

전처리 버전 가이드

전처리 버전 가이드 — 팀원용

이 폴더에 뭐가 있나요?

같은 데이터를 **8가지 방식으로 전처리**한 CSV 파일입니다.

이상치 포함/제거 x 스케일링 4종 = 8버전. 모델에 따라 골라서 쓰면 됩니다.

1. 모든 버전에 공통으로 적용된 처리

단계	처리 내용	예시
결측치 (구조적)	0으로 채움 + 플래그 유지	rating -> 0, rating_na=True
결측치 (랜덤)	train 중앙값으로 채움	hr_rest_days 누락 -> 비슷한 말들의 중간값
중복 피처 제거	상관계수 0.8+ 쌍에서 하나 삭제	chaksun2~5 삭제 (chaksun1만 유지)
데이터 분할	시간순 6:2:2 (train/valid/test)	과거 60% 학습, 최근 20% 평가

핵심 코드:

```
# Structural missing -> fill with 0
structural_cols = ["rating", "hr_winrate", "hr_plcrate", ...]
for col in structural_cols:
    df[col] = df[col].fillna(0)

# Random missing -> train median
medians = df.loc[train_mask, remaining_cols].median()
df[remaining_cols] = df[remaining_cols].fillna(medians)

# Drop high-correlation features
drop_cols = ["chaksun2", "chaksun3", "chaksun4", "chaksun5", "buga2",
```

```
"buga3", ...]
df = df.drop(columns=drop_cols)

# Time-based split 6:2:2
df["fold"] = assign_time_split(df, date_col="rcDate", ratios=(0.6, 0.2, 0.2))
```

2. 이상치 처리 (v1~v4 vs v5~v8의 차이)

이상치란? 다른 값들과 동떨어진 극단값.

예) 평균 휴양일 42일인데 868일 쉰 말 = 이상치.

탐지 방법: IQR ($Q3 + 1.5 \times IQR$ 초과 또는 $Q1 - 1.5 \times IQR$ 미만)

버전	이상치 처리	언제 쓰나
v1~v4	이상치 포함 (원본 그대로)	트리 모델(RF, XGBoost)은 이상치에 강건
v5~v8	이상치 제거 (IQR 밖 행 삭제)	로지스틱회귀, KNN 등 스케일 민감 모델

핵심 코드:

```
# IQR outlier detection (fitted on train set only)
Q1 = train_df[col].quantile(0.25)
Q3 = train_df[col].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR

# Remove rows outside IQR (only if outlier rate <= 30%)
for col, info in outlier_info.items():
    if info["pct"] <= 30:
        mask &= (df[col] >= info["lower"]) & (df[col] <= info["upper"])
df_clean = df[mask]
```

이상치 제거 기준:

- 이상치 비율 30% 초과 피쳐는 분포 자체가 넓은 것으로 간주, 제거 안 함
- 배당률 관련 피쳐는 제외 (이미 model_features에서 빠져 있음)
- 상세 결과는 **results/eda/report.html**의 3장 참고

3. 스케일링이 뭔가요?

문제: "마체중"은 400~580 범위, "승률"은 0~1 범위. 로지스틱회귀 같은 모델은 숫자가 큰 컬럼에 더 영향을 받아 불공평해짐.

해결: 모든 컬럼의 숫자 범위를 비슷하게 맞춰주는 게 "스케일링".

방법	공식	특징
없음 (v1, v5)	원본 그대로	트리 모델은 스케일링 불필요
StandardScaler (v2, v6)	$(값 - 평균) / 표준편차$	결과 대략 -3~+3. 가장 보편적.
MinMaxScaler (v3, v7)	$(값 - 최소) / (최대 - 최소)$	결과 정확히 0~1. 거리 계산에 좋음.
RobustScaler (v4, v8)	$(값 - 중앙값) / IQR$	극단값에 덜 흔들림.

핵심 코드:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
```

```
scaler = StandardScaler()          # or MinMaxScaler() or RobustScaler()
scaler.fit(df.loc[train_mask, cols]) # fit on train only!
df[cols] = scaler.transform(df[cols]) # apply same transform to all
```

4. 전체 버전 한눈에 보기

파일	이상치	스케일링	추천 모델
v1_base.csv	포함	없음	RF, XGBoost
v2_standard.csv	포함	Standard	로지스틱, SVM
v3_minmax.csv	포함	MinMax	KNN, K-means
v4_robust.csv	포함	Robust	이상치 강건
v5_base_no_outlier.csv	제거	없음	RF (정제)

파일	이상치	스케일링	추천 모델
v6_standard_no_outlier.csv	제거	Standard	로지스틱 (정제)
v7_minmax_no_outlier.csv	제거	MinMax	KNN (정제)
v8_robust_no_outlier.csv	제거	Robust	비교용

모르겠으면?

- 트리 모델(RF, XGBoost) -> v1_base.csv
- 로지스틱회귀 -> v6_standard_no_outlier.csv
- 여러 모델 비교 -> v1 vs v5로 이상치 영향 비교
- KNN/K-means -> v7_minmax_no_outlier.csv

5. 사용법

버전 파일을 직접 로드:

```
import pandas as pd
```

```
df = pd.read_csv("data/versions/v6_standard_no_outlier.csv", encoding="utf-8-sig")
```

```
train = df[df["fold"] == "train"]
valid = df[df["fold"] == "valid"]
test = df[df["fold"] == "test"]
```

```
y_train = train["win"]
# X: drop ID/market/outcome columns (see src/pipeline/config.py
EXCLUDE_COLS)
```

또는 이미 분리된 파일 사용:

```
train = pd.read_csv("data/splits/v6_standard_no_outlier/train.csv",
encoding="utf-8-sig")
valid = pd.read_csv("data/splits/v6_standard_no_outlier/valid.csv",
encoding="utf-8-sig")
test = pd.read_csv("data/splits/v6_standard_no_outlier/test.csv", encoding="utf-8-sig")
```

6. 주의사항

절대 하면 안 되는 것

- fold 컬럼을 무시하고 데이터를 섞는 것 -> 미래 정보 유출
- valid/test로 스케일링 기준을 잡는 것 -> 이미 train 기준으로 처리됨
- win 컬럼을 피쳐(X)로 사용 -> 이건 정답(y)임

KHUDA 3조 · 전처리 버전 가이드 · 2026-08-16 21:39